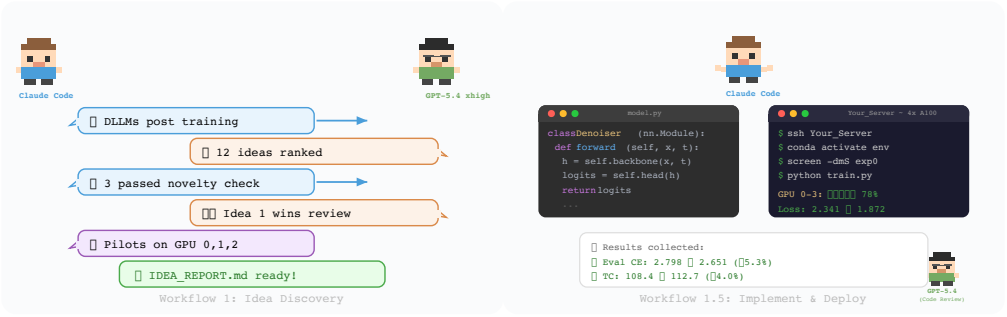
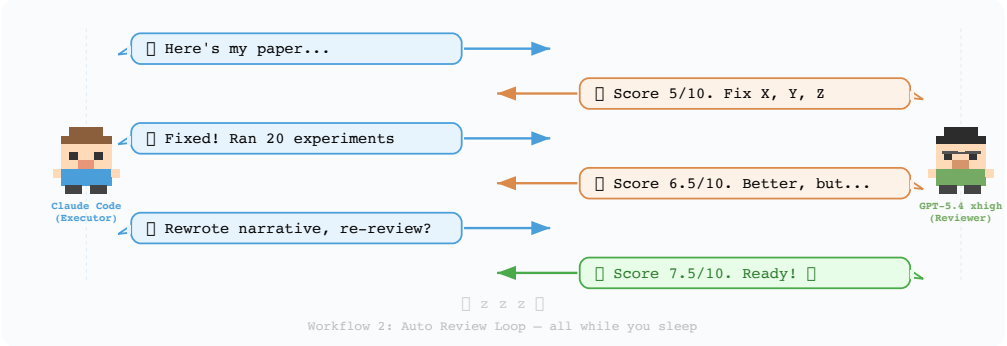
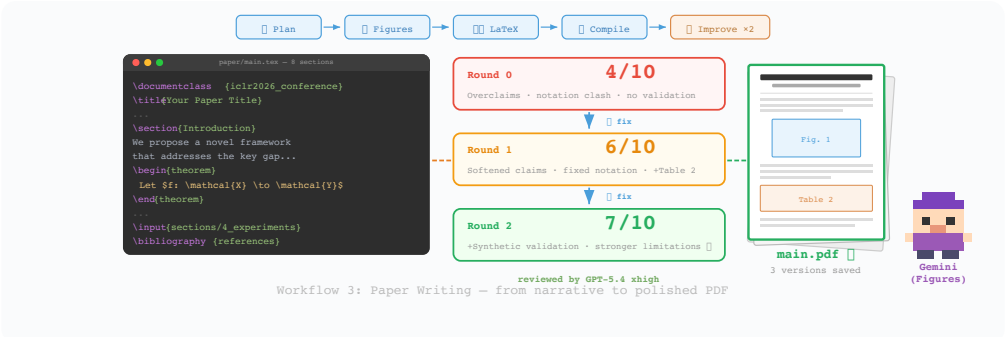
ARIS: Autonomous Research via Adversarial Multi-Agent Collaboration

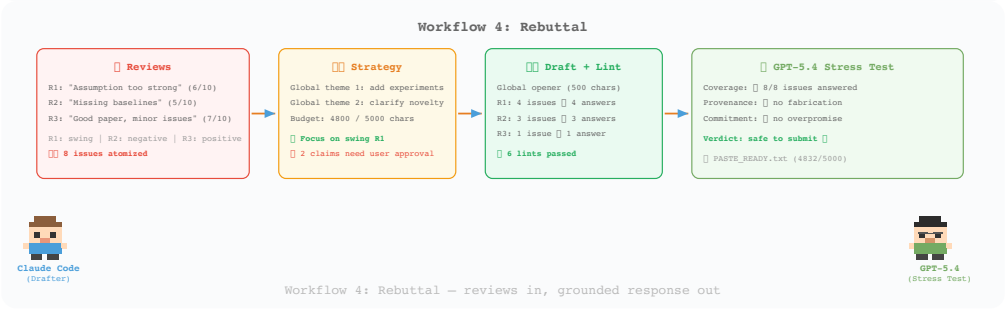
Ruofeng Yang[1†], **Yongcan Li**[1], **Shuai Li**[1,2*]
[1]Shanghai Jiao Tong University [2]Shanghai Innovation Institute
{wanshuiyin, joseph_y, shuaili8}@sjtu.edu.cn
[†]Project Leader [*]Corresponding Author

Project page: https://github.com/wanshuiyin/Auto-claude-code-research-in-sleep









ABSTRACT

This report describes ARIS(**A**utonomous **R**esearch via Adversarial Multi-Agent Collaboration), an open-source research harness for autonomous ML research, including its architecture, assurance mechanisms, and early deployment experience.The performance of agent systems built on large language models depends on both model weights and the *harness* around them, which is the system logic that governs what information to store, retrieve, and present to the model. For long-horizon research workflows, the central failure mode is not visible breakdown but *plausible unsupported success*: a long-running agent can produce claims whose evidential support is incomplete, misreported, or silently inherited from the executor's framing. Therefore, we present ARIS as a research harness that coordinates machine-learning research workflows through cross-model adversarial collaboration as a default configuration: an executor model drives forward progress while a reviewer from a different model family is recommended to critique intermediate artifacts and request revisions. ARIS has three architectural layers. The *execution layer* provides more than 65 reusable Markdown-defined skills, model integrations via MCP, a persistent research wiki for iterative reuse of prior findings, and deterministic figure generation. The *orchestration layer* coordinates five end-to-end workflows with adjustable effort settings and configurable routing to reviewer models. The *assurance layer* includes a three-stage process for checking whether experimental claims are supported by evidence—integrity verification, result-to-claim mapping, and claim auditing that cross-checks manuscript statements against the claim ledger and raw evidence—as well as a five-pass scientific-editing pipeline, mathematical-proof checks, and visual inspection of the rendered PDF. A prototype self-improvement loop records research traces and proposes harness improvements that are adopted only after reviewer approval.

1 INTRODUCTION

Recent work on harness engineering (Lee et al., 2026) suggests that the performance of LLM systems can depend heavily on the *harness*—the surrounding system logic that governs storage, retrieval, and presentation—as well as on model weights. Machine-learning research poses an unusually complex harness-engineering problem: the workflow spans literature review and hypothesis generation through experimentation, internal critique, manuscript preparation, and responses to external feedback. This research harness is still assembled manually in many settings: researchers coordinate compute, references, manuscript tooling, and feedback workflows across separate systems (Lu et al., 2024; Schmidgall et al., 2025).

Several autonomous research agents now target specific parts of this workflow. The AI Scientist (Lu et al., 2024) and AI Scientist v2 (Yamada et al., 2025) automate a pipeline from idea generation to manuscript drafting. Agent Laboratory (Schmidgall et al., 2025) adds human-in-the-loop checkpoints to the workflow. These systems exhibit three recurring limitations that motivate our design: (1) many rely on the same or closely related model family for both execution and review—a same-model self-refinement pattern in the spirit of Madaan et al. (2023); Shinn et al. (2024)—which can leave correlated errors uncaught when generator and validator share inductive biases (an effect that motivates work on heterogeneous multi-agent debate Du et al., 2024; Liang et al., 2024a); (2) workflows are tightly coupled end-to-end, making it difficult to replace individual stages or resume from saved intermediate states; (3) few provide explicit, system-level checks on experimental integrity and manuscript quality.

As current agents become more capable of carrying out long-horizon tasks, it is possible to conduct fully autonomous research from an intuition or a basic idea. However, when using a single agent to conduct a long-term hard task, it may exhibit laziness, hallucinations, or deceptive behavior. The central risk for an autonomous research harness is not only outright failure, but *plausible unsupported success*: results may be real yet misreported, claims may

outrun the evidence that licenses them, and downstream readers may silently inherit the executor's framing. Hence, we propose the following stringent assumption:

Any long-term task performed by a single agent is unreliable.
We need to divide the total workflow into sub-workflows and cross-family models to review the output at each step independently.

This assumption may understate the capabilities of current agents, but the trade-off favors strictness in a high-rigor field like research: an adversarial reviewer offers a clear quality gain even though adversarial review introduces a harder optimization problem for the executor. Think of it as adversarial vs. stochastic bandits—a single model self-reviewing is the stochastic case (predictable reward noise), while cross-model review is adversarial (the reviewer actively probes weaknesses the executor did not anticipate), and adversarial bandits are fundamentally harder to game. Two agents (executor and reviewer) are also the minimum needed to break self-play blind spots, and two-player games converge to a Nash equilibrium far more efficiently than n-player ones.

This stringent assumption decomposes operationally into three bottlenecks. First, *persistent research state* (i) is required because stepwise review is meaningless if the system cannot preserve the artifacts, decisions, evidence, and claims that connect one sub-workflow to the next. Second, *modular execution* (ii) is required because a long research trajectory must be divided into replaceable stages rather than hidden inside a single opaque agent trajectory. Third, *independent assurance* (iii) is required because the reviewer must not merely continue the executor's reasoning, but examine the produced artifact from a sufficiently different model family, context policy, or audit role. These are not separate desiderata added after the fact; they are the system-level consequences of treating single-agent long-horizon research as unreliable by default.

ARIS responds by treating assurance as a first-class workflow layer rather than a single review pass, separating artifact production from evidence checking, claim mapping, and manuscript review. Concretely, reusable Markdown-defined skills are coordinated under a default cross-family executor/reviewer pairing, with explicit assurance checks at key experimental and manuscript stages. We default to cross-family pairings because prior work suggests that mixed-model agent configurations can produce less correlated and more varied critiques (Du et al., 2024; Liang et al., 2024a); we adopt this as a recommended configuration rather than a hard system constraint.

We describe three aspects of ARIS:

1. An **assurance stack** that uses separate executor and reviewer models, including a three-stage process for checking whether claims are supported by evidence (integrity verification, result-to-claim mapping, claim auditing against the claim ledger and raw evidence), a five-pass scientific-editing pipeline, mathematical-proof checks, and visual PDF inspection (§3).
2. A **modular system architecture** organized into three layers—execution, orchestration, and assurance—with more than 65 reusable skills, a persistent research wiki for iterative reuse of prior findings, deterministic figure generation, adjustable effort levels, configurable reviewer routing, and a prototype self-improvement loop (§2–§4.5).
3. **Early deployment experience** across three tested executor platforms with adaptation guides for three additional platforms, including community usage reports and an analysis of current limitations (§5).

Remark 1 (Discussion of Human in the Loop). Though ARIS is an auto-research system, we still note that human-in-the-loop can significantly improve the generation quality of final papers and can help users to gain more knowledge of writing papers, which is essential for cultivating one's research taste.

2 System Overview

Following the harness-engineering taxonomy of Lee et al. (2026), ARIS is a *research harness*: a stateful system that orchestrates interactions with LLMs by selecting the context, tools,

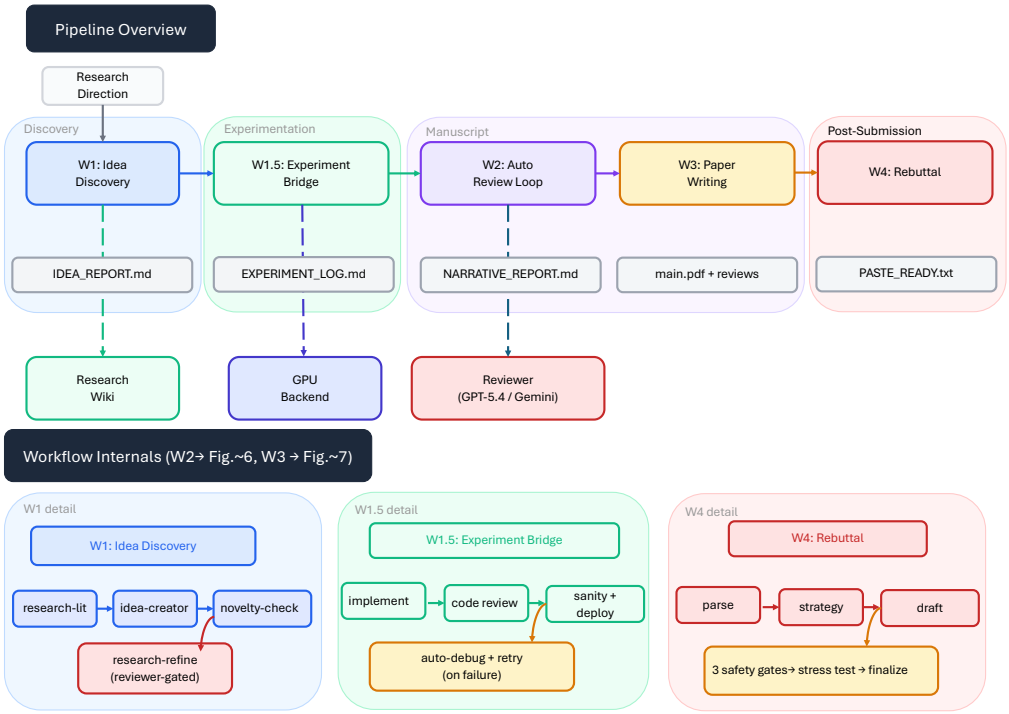


Figure 1: ARIS workflow library. **Top**: end-to-end composition of the five workflows and their artifact contracts, grouped into four research phases (Discovery, Experimentation, Manuscript, Post-Submission); dashed links denote reviewer feedback, GPU-triggered evidence collection, and wiki memory. **Bottom**: compressed internal structure for the workflows not otherwise expanded in the main text—W1 idea discovery (with reviewer-gated refinement), W1.5 experiment bridge (with code review and auto-debug fallback), and W4 rebuttal (with safety gates and stress test). W2 auto-review and W3 paper writing internals are detailed separately in Figures 2 and 3.

Table 1: Current ARIS implementation footprint (v0.4, April 2026).

Component	**Scope**
Skills	More than 65 Markdown-defined files
Workflows	5 end-to-end (+ full pipeline command)
Model bridges	6 MCP bridges (Codex, Oracle, Claude, Gemini, MiniMax, llm-chat)
Tested executors	3 (Claude Code, Codex CLI, Cursor); 3 adapted
Assurance stack	3-stage audit cascade + manuscript quality
Persistent memory	Per-project research wiki (4 entity types)
Effort presets	4 levels (lite / balanced / max / beast)
Dependencies	None for skills; single binary for CLI

and feedback presented to them during each stage of a research workflow. Before describing how the harness is organized internally, we first summarize what it does end-to-end. Figure 1 shows the workflow library: five workflows—idea discovery, experiment bridge, auto-review, paper writing, and rebuttal—chained through plain-text artifact contracts and grouped into four research phases (Discovery, Experimentation, Manuscript, Post-Submission). Figures 2 and 3 zoom into the two assurance-heavy workflows revisited when describing workflow orchestration in §4: Workflow 2 (Auto Review Loop) and Workflow 3 (Paper Writing). The architecture, design principles, and adversarial-collaboration mechanism that realize these workflows are described in the remainder of this section; per-skill details follow in §4.

Figure 4 illustrates the three-layer architecture, and Table 1 summarizes the implementation described in this report.

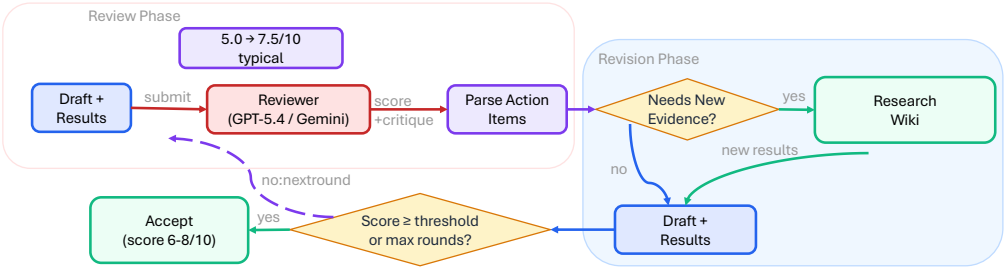


Figure 2: Workflow 2: Auto Review Loop. Each round submits the draft to a cross-model reviewer for structured scoring, extracts action items, optionally runs GPU experiments for new evidence, revises affected sections, and checks convergence. The loop terminates when the score exceeds a predefined threshold or after a preset maximum of rounds.

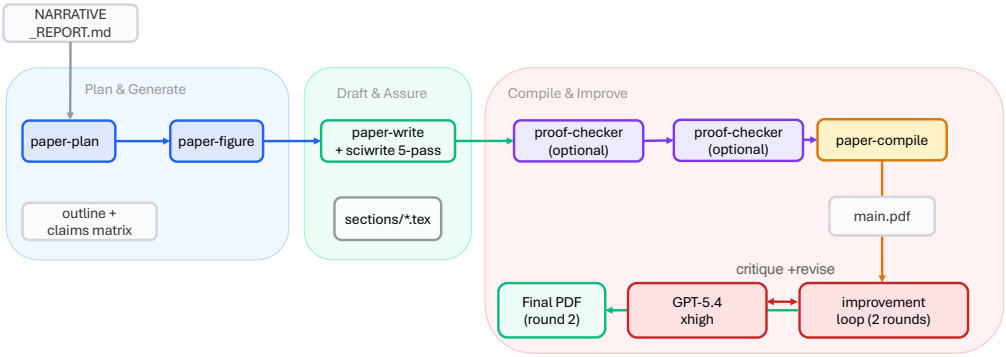


Figure 3: Workflow 3: Paper Writing Pipeline. Three phases: *Plan & Generate* (outline, figures), *Draft & Assure* (LaTeX drafting with five-pass editing, optional proof checking, claim auditing), and *Compile & Improve* (compilation, two rounds of GPT-5.4 xhigh visual review with automatic revision).

These layers map to the three bottlenecks identified in §1: persistent state (i) is realized by the per-project research wiki and versionable artifact contracts described in §4.2; modular execution (ii) is realized by self-contained Markdown skill files coordinated through the workflows of Figure 1; and independent assurance (iii) is realized by the assurance layer (§3) under the cross-family executor/reviewer pairing detailed below.

2.1 Design Principles

The design of ARIS is guided by five principles. Principles (1), (3), and (5) instantiate bottlenecks (iii), (ii), and (i) respectively from §1; principle (2) is the implementation choice that makes (ii) ergonomic, and principle (4) is the engineering constraint that lets these controls survive across executor environments.

(1) Heterogeneous models over single-model self-refinement. Single-model self-refinement loops (Madaan et al., 2023; Shinn et al., 2024) have generator and validator that share inductive biases; heterogeneous multi-agent debate has been reported to elicit more diverse critiques than homogeneous configurations (Liang et al., 2024a; Du et al., 2024). ARIS *defaults to* pairing executor and reviewer from different model families and treats this as the recommended configuration. Here, a *model family* denotes a shared model lineage or provider class (e.g., Claude models form one family; GPT models form another). The default configuration we ship and document is Claude-family executor with GPT-family reviewer (Codex MCP, Oracle MCP) or vice versa; users can also configure Gemini or MiniMax through dedicated MCP bridges, and GLM, Kimi, or DeepSeek as the reviewer through the generic OpenAI-compatible `llm-chat` bridge listed in Table 1.

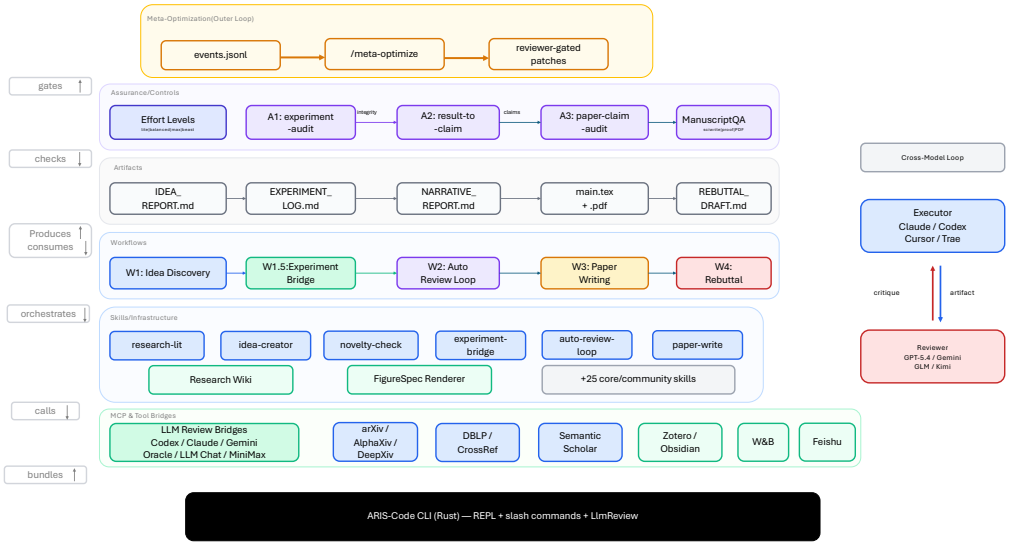


Figure 4: ARIS system topology. Six component groups interact through labeled relationships (left margin): the **Meta-Optimization** outer loop gates the **Assurance** layer, which checks **Artifacts**; artifacts are produced and consumed by **Workflows**, which orchestrate **Skills**; skills call **MCP & Tool Bridges** for external model and data access. The executor and reviewer (right) use models from different families. **ARIS-Code CLI** bundles all components into a standalone binary.

(2) Modular skill files over monolithic agents. Each research capability is defined primarily by a `SKILL.md` file, a plain-text Markdown specification that can be interpreted by multiple LLM-based coding agents, enabling independent development, domain-specific extensions, and component-level updates.

(3) Composability over fixed pipelines. Skills can be chained into workflows, with per-invocation parameter overrides and checkpoint-based recovery across sessions.

(4) Portability over vendor lock-in. The skill library is distributed as plain-text files and does not depend on a platform-specific runtime; in our current setup, the same `SKILL.md` files can be used in Claude Code, Codex CLI, and Cursor with no file-level changes.

(5) Persistent memory over ephemeral context. Each project maintains a research wiki that stores papers, ideas, experiment records, and tracked claims across sessions, allowing the system to revisit and refine prior work rather than restarting from a stateless prompt each session (Karpathy, 2026).

2.2 Cross-Model Adversarial Collaboration

The core mechanism is a *critique-to-action loop*. The executor first produces an artifact (code, manuscript section, or experiment design). A reviewer—which the recommended configuration draws from a different model family—then assigns a review score under a predefined rubric and returns structured action items. The executor addresses those items, after which a convergence check decides whether to run another round or accept the artifact as provisionally satisfactory. The loop terminates either when the review score exceeds a predefined threshold (default 6/10) and all critical review items have been resolved, or when it reaches a preset maximum number of rounds (default 4).

Reviewer independence. The executor supplies file paths and a review objective. The reviewer then reads the referenced artifacts directly and forms an independent assessment. If the executor first summarized the artifact, the reviewer would assess the executor's framing

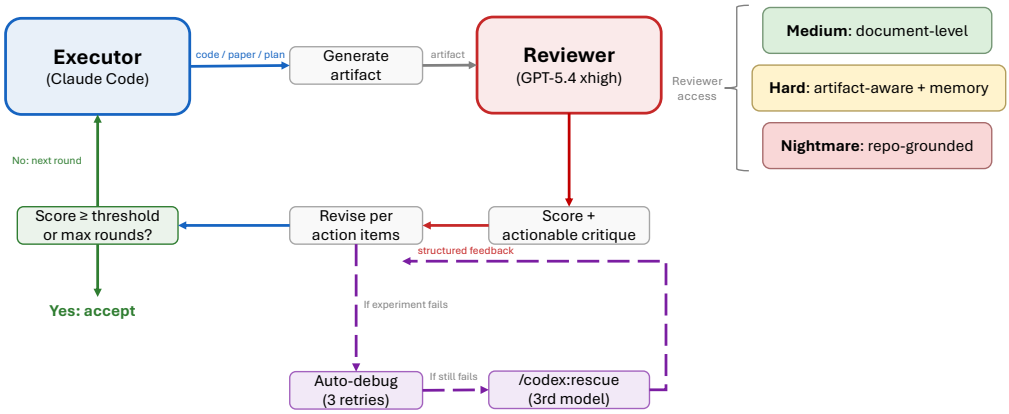


Figure 5: Cross-model adversarial collaboration alternates executor generation with external-model critique, actionable revision requests, and convergence checking. Reviewer access ranges from document-only to repository-level.

rather than the underlying work, thereby increasing the risk of shared errors. This protocol is specified in a shared protocol document that every skill invoking a review step must follow.

Reviewer access and context policy. ARIS configures reviewers along two orthogonal axes. The first axis is *access scope*: *document-only* (reviewer reads the manuscript text), *artifact-augmented* (reviewer additionally reads supporting artifacts such as result files), and *repository-level* (reviewer directly inspects the codebase and generated outputs through repository access tools). The second axis is *context policy*: *fresh* (each review round opens a new thread with no prior context, used to prevent confirmation bias) versus *cross-round* (reviewer retains state across rounds and explicitly verifies whether previously raised issues have been addressed). Appendix C defines each axis in detail and notes which axis settings are required by specific assurance skills.

Automatic debugging and fallback diagnosis. When experiments fail, the system assigns the failure to a predefined error class, applies a class-specific remediation, and retries up to a configurable limit (default three attempts). The executor must attempt at least two distinct remediation strategies before marking a reviewer issue as unresolved. If both remediation attempts fail, a third, independently configured model can provide an independent diagnosis through a dedicated rescue step.

3 Cross-Model Assurance Stack

The adversarial collaboration described in §2.2 provides a general critique loop. It seems perfectly natural that the executor agent only needs to communicate adversarially with the reviewer agent based on the article's content. However, the reality is much more complex. To improve the peer review score as quickly as possible, the executor agent will use various methods to deceive the reviewers during the dialogue. Therefore, we need to set up a strict assurance stack.

This section presents the *assurance stack* that ARIS adds to the critique loop as its operational response to bottleneck (iii) of §1 and to the *plausible unsupported success* risk introduced there: a three-stage evidence-to-claim audit cascade for experimental integrity (§3.1), a manuscript assurance layer for prose, proof, and presentation quality (§3.2), and two system-wide controls—effort levels and reviewer routing—that set audit depth and reviewer backend (§3.3).

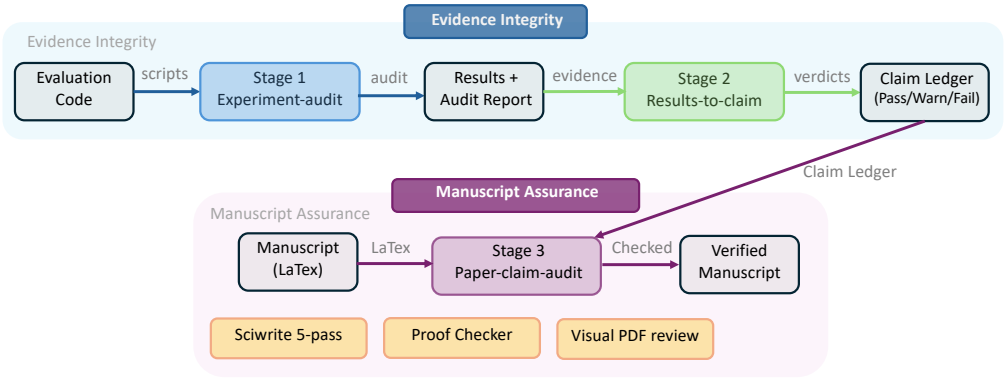


Figure 6: Evidence-to-Claim Audit Cascade. **Stage 1** (`experiment-audit`): the reviewer audits evaluation scripts and result files for integrity failure modes. **Stage 2** (`result-to-claim`): results are mapped to explicit claim verdicts (supported, partial, invalidated); claims with audit failures are downgraded. **Stage 3** (`paper-claim-audit`): a zero-context fresh reviewer compares every quantitative claim in the manuscript against the claim ledger and raw result files. The **Manuscript Assurance** layer applies four components: a five-pass editing pipeline, proof verification, visual PDF review, and citation-audit (verifying every `\cite` for existence, metadata correctness, and context appropriateness).

3.1 Evidence-to-Claim Audit Cascade

Community reports and internal debugging revealed that executor agents can produce misleading experimental outputs, including model-derived references, self-normalized metrics, and claims unsupported by output files. ARIS addresses these failure modes with a three-stage audit pipeline (Figure 6). Stage 1 audits evaluation integrity, Stage 2 maps results to explicit claims, and Stage 3 independently verifies manuscript claims against the source and raw evidence using a reviewer that the recommended configuration draws from a model family different from the executor's.

Stage 1: Experiment-integrity audit (`/experiment-audit`). A cross-model reviewer audits the evaluation code and outputs against the following integrity failure modes: (1) *model-derived reference labels*—reference targets are synthesized from model outputs rather than obtained from the dataset or another declared source; (2) *self-normalized scores*—metrics use denominators derived from the model's own predictions, which can inflate or distort reported performance; (3) *phantom results*—claimed numbers that do not match actual output files; (4) *dead-code or unused-metric inflation*—evaluation code defines additional metrics or branches that are never executed but are described as part of the analysis; (5) *scope inflation*—claims generalize beyond the tested datasets, seeds, or experimental settings. The audit produces a structured report (`EXPERIMENT_AUDIT.md`) and a machine-readable JSON summary. The audit is advisory at the workflow level: it does not halt execution, but downstream stages propagate warning or failure statuses into later claim judgments.

Stage 2: Result-to-claim mapping (`/result-to-claim`). Each candidate experimental claim is evaluated against the available evidence and assigned one of three verdicts: *supported*, *partially supported*, or *invalidated*. If a Stage 1 audit report is available, its `integrity_status` is propagated to each claim record; claims with `fail` cannot be marked fully supported until the integrity issue is resolved. The output is a *claim ledger* that maps each experimental claim to the evidence that supports, qualifies, or contradicts it.

Stage 3: Paper-claim audit (`/paper-claim-audit`). A fresh zero-context reviewer—implemented as a new Codex thread with no prior conversation history—reads the manuscript LaTeX source together with raw result and configuration files, then cross-checks the paper's quantitative claims. This fresh-thread design reduces the risk that prior executor context or accumulated reviewer expectations bias the audit. Representative checks include numerical

mismatches, best-seed cherry-picking, configuration mismatches between the manuscript and experiment files, aggregation or delta-arithmetic errors, and scope overclaim. Each claim receives a structured audit status such as `exact_match`, `rounding_ok`, `number_mismatch`, `config_mismatch`, or `missing_evidence`.

Conceptually, the stages move from code-level integrity, to evidence-to-claim interpretation, to manuscript-level reporting fidelity. Each stage can be invoked independently. In the full research pipeline, Stage 1 runs after experiments, Stage 2 assembles claim records from results, and Stage 3 is used during paper writing and final manuscript review.

3.2 Manuscript Assurance

Beyond evidence integrity, ARIS adds four mechanisms for manuscript assurance.

Five-pass scientific-editing pipeline. Inspired by the principles of scientific writing pedagogy (Sainani, 2019), the `/paper-write` skill applies five automated editing passes after initial drafting: (1) *Clutter removal*: remove filler phrases, redundant words, and unnecessary hedging; (2) *Active voice*: convert passive constructions to active where appropriate; (3) *Sentence structure*: improve topic positioning and local coherence without forcing a single sentence template; (4) *Terminology consistency*: if the Methods section introduces a term such as "validation split," later sections should use the same term rather than an informal variant—extract domain-specific key terms and verify consistent usage across sections; (5) *Numerical consistency*: cross-check repeated numerical statements against the corresponding table, figure, or cited result file.

Proof verification (`/proof-checker`). For theory-heavy papers, the proof-checker uses a 20-category issue taxonomy together with a two-axis severity scheme that separates proof status (e.g., invalid, unjustified, unclear) from impact (global, local, cosmetic). The checker verifies theorem applications against side-condition checklists and runs a counterexample red-team pass on key lemmas and major guarantees. The output is a *proof-obligation ledger* that records the verification status of each theorem, lemma, and derived obligation.

Visual PDF review. The `/auto-paper-improvement-loop` sends *both* the LaTeX source and the compiled PDF to the reviewer. The reviewer assesses substantive content from the source and visual presentation from the PDF: figure readability, caption–figure alignment, layout quality (orphaned headers, misplaced floats), table formatting, and color consistency across all figures. This dual-input review catches presentation issues that source-only review misses.

Citation audit (`/citation-audit`). The fourth manuscript-assurance component verifies every `\cite` in the paper along three independent axes: (i) *existence*—the cited paper resolves at the claimed arXiv ID, DOI, or venue; (ii) *metadata correctness*—author names, year, venue, and title match canonical sources (DBLP, arXiv, ACL Anthology, Nature, OpenReview); (iii) *context appropriateness*—the cited paper actually establishes the claim it is being used to support. The third axis is the most diagnostic: a real paper used to support a wrong claim is a credibility failure that metadata-only checks miss. Verification uses fresh cross-family reviewers with web access; verdicts are recorded in a per-entry ledger and surfaced as KEEP/FIX/REPLACE/REMOVE recommendations for human approval before submission.

3.3 Effort Levels and Reviewer Routing

Effort levels. ARIS exposes four effort presets that scale breadth-, depth-, and iteration-related settings while leaving core review invariants unchanged: `lite` ($\approx 0.4\times$) reduces the number of papers surveyed, ideas generated, and review rounds for quick exploration; `balanced` ($1\times$, default) provides standard behavior; `max` ($\approx 2.5\times$) increases search depth, review thoroughness, and experiment repetitions; `beast` (≈ 5–$8\times$) pushes breadth- and iteration-related settings toward their upper bounds. Users can override the default with an inline directive such as `effort: max`. A key invariant is that Codex-based reviewer calls

use `xhigh` reasoning effort regardless of the overall effort preset, so effort scaling changes coverage and iteration counts rather than the reviewer's reasoning budget.

Reviewer routing. In the current implementation, review requests route to GPT-5.4 via the Codex MCP bridge. For especially high-stakes reviews, users can explicitly route supported skills to GPT-5.4 Pro via the Oracle MCP bridge with an inline directive such as `reviewer: oracle-pro`. In the current implementation, Oracle routing is enabled for a subset of reviewer-invoking skills. Alternative reviewer backends can also be connected through the `llm-chat` bridge, subject to the same reviewer-independence protocol and the recommendation that reviewer and executor come from different model families (§2).

4 Implementation: Skills, Workflows, and Tools

The assurance layer is covered in §3. This section describes the implementation of the execution and orchestration layers: the skills layer that breaks long research trajectories into inspectable, replaceable units—ARIS's answer to the modular-execution bottleneck (ii) of §1 (§4.1); a per-project research wiki that addresses the persistent-state bottleneck (i) (§4.2); workflow orchestration (§4.3); and supporting tools (§4.4); it then discusses a prototype meta-optimization outer loop (§4.5).

4.1 Skills Layer

The foundation of ARIS is a library of more than 65 research-oriented skills (Appendix B), each encoded as a single `SKILL.md` file. A `SKILL.md` contains a YAML frontmatter (name, description, trigger conditions, allowed tools) followed by a natural-language workflow specification: inputs, outputs, step-by-step procedures, quality gates, and failure-handling instructions. Skills range from simple utilities such as `/arxiv`, which retrieves paper metadata, to multi-step workflows such as `/auto-review-loop`, which iteratively reviews, revises, and, when needed, runs follow-up experiments.

Five shared reference documents provide cross-cutting guidance: `reviewer-independence.md`, `experiment-integrity.md`, `effort-contract.md`, `citation-discipline.md`, and `writing-principles.md`. Any skill can reference these; they codify system-wide invariants without duplicating rules across skill files.

Skills exchange intermediate artifacts through versionable text files and structured Markdown pages. For example, `IDEA_REPORT.md` is produced during idea discovery and consumed by `experiment-bridge`; `EXPERIMENT_LOG.md` is consumed by `auto-review-loop`; and `NARRATIVE_REPORT.md` is consumed by `paper-writing`. This design improves auditability, checkpoint-based recovery, and portability across model backends. Together, single-file skills and plain-text artifact contracts are how ARIS discharges bottleneck (ii) of §1: the long research trajectory is broken into inspectable, replaceable invocations whose inputs and outputs can be reviewed independently rather than hidden inside a single opaque agent transcript.

4.2 Research Wiki: Persistent Project Memory

ARIS realizes bottleneck (i) of §1—persistent research state across long-running, multi-session workflows—through four layered mechanisms: (1) the research wiki described in this subsection, which records papers, ideas, experiments, and claims as a structured knowledge graph; (2) the plain-text artifact contracts exchanged between skills (§4.1), which carry intermediate state across skill invocations; (3) a *file-system-as-state* design choice (Design Principle 5 of §2) that places all session state in versionable text files rather than in-memory caches or external databases, so any new session can pick up from the artifacts of a previous one; and (4) checkpoint-based recovery (Design Principle 3 of §2), in which any workflow can resume from the saved artifacts of an earlier run. The wiki is the headline component and is described next; the other three mechanisms are referenced where relevant.

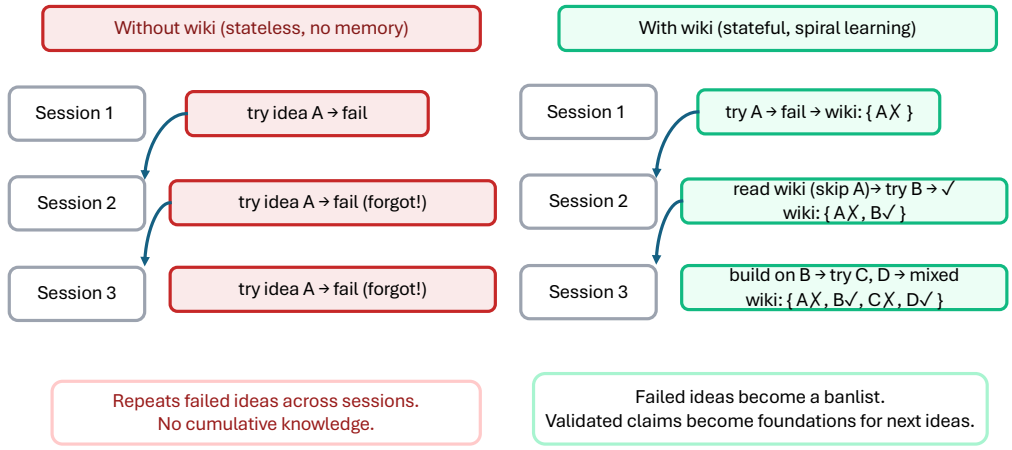


Figure 7: Why the wiki matters. **Without wiki** (left), each session starts from a blank slate; the same failed idea A can be re-tried indefinitely because the system has no memory of prior outcomes. **With wiki** (right), Session 1's failure is recorded; Session 2's ideation reads the wiki, skips A, and tries B successfully; Session 3 builds on B and explores C/D. Failed ideas become a banlist; validated claims become foundations for the next ideation round, converting one-shot research into spiral learning.

The research wiki provides persistent, cross-session memory through four entity types—papers, ideas, experiments, and claims—stored as structured Markdown pages with canonical node IDs. Eight typed relationships (extends, contradicts, addresses_gap, inspired_by, tested_by, supports, invalidates, supersedes) form a lightweight knowledge graph.

Three skills integrate with the wiki: `/research-lit` ingests discovered papers as structured pages; `/idea-creator` reads a compressed `query_pack.md` summary (capped at 8,000 characters) before ideation, using listed gaps as search seeds and previously rejected ideas to avoid revisiting unpromising directions; `/result-to-claim` updates claim status after each experiment. The key design choice is to retain rejected ideas: without persistent memory, an ideation pipeline can re-propose the same dead-end direction across sessions; with the wiki, the same direction is recognized as previously explored and the search moves on (Figure 7).

4.3 Workflow Orchestration

Five workflows chain skills into end-to-end pipelines. The overall composition is shown earlier in Figure 1 (§2); Table 2 lists inputs, outputs, and key skills; full appendix figures for all five workflows are also in Appendix A.

Auto-review loop (Workflow 2). In each round (Figure 2, §2), the draft is sent to a reviewer model from a different family for structured scoring; the system extracts actionable items, runs follow-up experiments when new evidence is requested and execution is permitted, revises affected sections, and resubmits the manuscript for review. The loop runs for up to four rounds or until the reviewer score exceeds a configurable threshold. One documented overnight run is described in §5.

Paper writing pipeline (Workflow 3). This workflow (Figure 3, §2) incorporates the assurance components described in §3. The pipeline currently chains seven core sub-skills, with `/proof-checker` invoked for theory-heavy papers: `/paper-plan` produces a structural outline and claims-evidence matrix; `/paper-figure` generates manuscript-ready figures and comparison tables; `/paper-write` drafts sections in LaTeX with citation lookup and a five-pass revision routine; optional `/proof-checker` audits theory-heavy sections; `/paper-claim-audit` performs an independent numerical consistency check; `/paper-compile` runs multi-pass compilation and repairs common LaTeX errors; and `/auto-paper-improvement-loop` performs two rounds of reviewer-model critique followed by revision. Users can invoke the full writing

Table 2: ARIS workflow library. Each workflow chains reusable skills through plain-text artifact contracts. For the idea discovery, the research taste is important for the idea quality, and we recommend the idea taste models provided by Tong et al. (2026). For experiments, users seeking SoTA results may also find AutoSoTA (Li et al., 2026) helpful.

Workflow	Input	Output	Key Skills
1. Idea Discovery	Research direction	Ranked idea report	`research-lit`, `idea-creator`, `novelty-check`, `experiment-plan`
1.5. Experiment Bridge	Experiment plan	Running code + results	`experiment-bridge`, `run-experiment`, `monitor-experiment`
2. Auto Review Loop	Draft + results	Improved paper	`auto-review-loop`, `research-review`, `analyze-results`
3. Paper Writing	Narrative report	Compiled PDF	`paper-plan`, `paper-figure`, `paper-write`, `proof-checker`, `paper-claim-audit`, `paper-compile`, `auto-paper-improvement-loop`
4. Rebuttal	Paper + reviews	Paste-ready rebuttal	`rebuttal` (7-phase pipeline with 3 safety gates)

stack through `/research-pipeline`; setting `auto_write: true` feeds Workflow 2 outputs directly into Workflow 3.

4.4 Tooling

Model bridges. ARIS currently exposes six MCP bridges for executor and reviewer routing: dedicated bridges for Codex, GPT-5.4 Pro review, Gemini, Claude, MiniMax, and a generic OpenAI-compatible chat bridge. Additional tool bridges cover citation lookup (DBLP/CrossRef), literature search (Semantic Scholar), reference-library sync (Zotero/Obsidian), experiment tracking (W&B), and mobile notifications (Feishu).

FigureSpec renderer. ARIS includes `figure_renderer.py`, a renderer that converts structured JSON FigureSpec descriptions into SVG figures. The renderer handles shape-aware edge clipping (for rectangular, circular, elliptical, and diamond nodes), self-loops, curved edges, multi-line labels with CJK text width estimation, and comprehensive input validation. FigureSpec is designed so that LLM agents can generate the JSON programmatically; under a fixed renderer version and font configuration, the same FigureSpec yields the same SVG output. All architecture and workflow diagrams in this report were generated with this pipeline.

ARIS-Code CLI. Beyond skill-based integration into existing IDEs, ARIS-Code is a standalone Rust-based CLI built on `claw-code` (UltraWorkers, 2026) that bundles all skills as slash commands. It ships as a single binary with an interactive REPL, a setup wizard, five LLM providers, and a native `LlmReview` tool for cross-model critique (Appendix D).

4.5 Meta-Optimization

Workflows 1–4 optimize research artifacts using a fixed harness. Meta-optimization targets the harness itself: the skill prompts, default parameters, and convergence rules (Lee et al., 2026).

ARIS implements a prototype outer loop in three components: (1) *Passive event logging*: in the current prototype, Claude Code hooks record structured events to `.aris/meta/events.jsonl` during normal usage, including timestamps, tool names, success or failure, and parameter overrides, without requiring manual logging. (2) *Pattern analysis*: the `/meta-optimize` skill analyzes usage statistics—which parameters users override most (suggesting suboptimal defaults), which tools fail repeatedly, where review scores plateau—and proposes targeted patches to the relevant `SKILL.md` files. (3) *Reviewer-gated application*: each proposed patch is reviewed by GPT-5.4 xhigh; only proposals scoring at least 7/10 are

Table 3: Deployment footprint as of April 2026.

Dimension	Current Status
Executor platforms	3 tested + 3 adapted (6 total)
Reviewer models	6+ (GPT, Gemini, GLM, MiniMax, Kimi, DeepSeek)
GPU backends	4 (local, SSH, Vast.ai, Modal)
Venue templates	9 families
Free-tier API access	ModelScope (no paid API keys required)
Community contributions	30+ contributed skills across robotics, hardware, communications, math

surfaced to the user as recommended candidates. The user makes the final decision; ARIS never auto-applies harness changes.

5 Deployment Evidence and Limitations

We summarize deployment footprint and limitations together. All reported outcomes are *observational*; they cannot be causally attributed to ARIS alone.

5.1 Ecosystem and Adoption

Table 3 summarizes the current deployment footprint. At the time of writing, the skill library had grown from 21 core skills at initial release to more than 65 skills spanning robotics, hardware design, communications, mathematical proof, grant writing, and presentation generation. At the time of writing, three additional executor environments are documented through community-maintained adaptation guides hosted in external repositories.

To illustrate the auto-review loop's operational dynamics under realistic conditions, we documented one overnight run end-to-end. Over approximately eight hours, the system completed four review–revise rounds, increased an internal reviewer score from 5.0 to 7.5/10, launched more than 20 GPU experiments, and removed claims that were not supported by the available evidence. This is a single trajectory on one paper; we do not generalize from it.

This run should be read as evidence that the harness can operationalize claim pruning and review-driven revision in one realistic trajectory, not as causal evidence that cross-family review is superior to same-family review or that two cross-family reviewers are an optimal committee size. The bandit and game-theoretic framing in §1 is used as a design analogy that motivates the two-role pattern; isolating its effect from researcher expertise, model choice, and task difficulty requires the controlled benchmark protocol described in Appendix E as future work.

6 Limitations and Responsible Use

No guarantee of correctness. ARIS cannot guarantee that any output is correct, novel, or scientifically sound. LLM outputs can include factual hallucinations and methodological gaps; cross-model review reduces some failure modes without eliminating them. Citation grounding via DBLP and CrossRef reduces but does not eliminate bibliography fabrication; Section 4 describes the lookup procedure used in our paper-writing workflow.

Audit limitations. The three-stage audit cascade can catch common integrity failures, but it cannot detect every error, inconsistency, or fabrication. It is an advisory safety net, not a formal verification system.

Reviewer bias amplification. The review loop can amplify reviewer biases: if the reviewer consistently demands a particular methodology, the loop may overfit to the reviewer model's preferences rather than improve broader scientific quality. Over-iteration past diminishing returns can degrade paper quality.

Human responsibility. ARIS automates execution and review loops; humans provide research direction, validate evidence, and make final submission decisions. Configurable checkpoints (e.g., `human checkpoint: true`) can be used to require human approval at each workflow step.

Security. Repository-level review may send source code to external LLM APIs, raising confidentiality concerns. Users should not enable repository-level review on repositories containing sensitive code or secrets unless an approved local-only review path is available. Local-only reviewer routing is planned but not yet implemented.

Self-referential disclosure. ARIS assisted with drafting and review of this technical report, but the authors manually reviewed, edited, and accepted responsibility for all final content.

7 Related Work

Autonomous research systems. Prior autonomous research systems differ in scope. The AI Scientist (Lu et al., 2024) and AI Scientist-v2 (Yamada et al., 2025) pursue end-to-end idea-to-paper automation; AI co-scientist (Gottweis et al., 2025) emphasizes hypothesis generation; Agent Laboratory (Schmidgall et al., 2025) introduces human-in-the-loop checkpoints; and data-to-paper (Ifargan et al., 2025) targets annotated-data-to-paper workflows with human oversight, programmatic back-tracing, and human-verifiable, information-traceable manuscripts. These systems differ in how much research state they retain across sessions; some recent systems provide run-level checkpoints or shared research repositories for cumulative progress, such as AgentRxiv (Schmidgall et al., 2025). However, few expose a per-project, structured research memory that jointly records literature notes, ideas, experiments, negative outcomes, and claim status for reuse across sessions. In contrast, ARIS defaults to cross-family executor-reviewer separation, ships reusable Markdown skill specifications, maintains a per-project research wiki for persistent cross-session memory of papers, ideas, experiments, and tracked claims (§4.2), and targets portability across multiple executor platforms with limited platform-specific logic. Recent critical analyses of autonomous research systems (Luo et al., 2025) identify integrity failure modes such as inappropriate benchmark selection, data leakage, metric misuse, and post-hoc selection bias, motivating the explicit assurance machinery we describe in §3. Very recently, more interesting auto-research systems, for example, AutoResearchClaw (Liu et al., 2026) and EvoScientist (Lyu et al., 2026) have been built [1].

Self-refinement and multi-agent debate. Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2024) demonstrate iterative self-feedback and verbal reflection. Multi-agent debate (Du et al., 2024) has been reported to improve reasoning in some settings, while divergent-debate work (Liang et al., 2024a) highlights both the value of forcing alternative arguments and the complications introduced when heterogeneous LLMs participate in judging or debate. ARIS draws on these ideas by embedding cross-model review loops throughout the research workflow. The bandit and two-player game-theoretic language we use in §1 should be read as a design analogy rather than a formal regret or equilibrium result: same-model self-review resembles repeated evaluation under correlated noise, whereas an external reviewer introduces an adversarial role that searches for failure modes the executor did not anticipate. ARIS adopts the minimal two-role version of this idea to break self-review blind spots while avoiding the API cost and coordination overhead of larger reviewer committees.

Automated reviewing. ReviewerGPT (Liu & Shah, 2023) and large-scale analyses (Liang et al., 2024b) suggest that LLMs can assist targeted review tasks and produce feedback overlapping with human reviewers on some dimensions, while remaining unsuitable as complete substitutes for expert peer review. ARIS uses external-model review as a development

[1] Readers can find a broader catalog of auto-research systems at `https://cadslab.github.io/Pantheon/`.

Table 4: Feature comparison. Each column is operationally defined in the caption below; entries reflect features explicitly documented in the cited papers/repos as of our review (April 2026), not author judgment about overall system quality. *partial* denotes documented support for a narrower, non-default, or non-identical variant of the feature that does not satisfy the full operational definitions used here. †: tested on 3 platforms (Claude Code, Codex CLI, Cursor) with documented adaptation guides for 3 more. ‡: data-driven end-to-end workflow from annotated data to manuscript, rather than open-ended idea-to-paper research. **Cross-family policy**: whether the system enforces, defaults to, optionally supports, or does not address cross-family executor/reviewer separation. Entries: *required* (system refuses same-family configurations), *default* (recommended and shipped configuration is cross-family, but not enforced), *optional* (supported but not the default), *none* (no notion of family separation). **Adversarial review**: explicit reviewer-vs-executor critique loop with revision. **Composable skills**: workflows assembled from independently invocable, single-file skill specifications. **E2E research workflows**: covers idea → experiment → paper end-to-end. **Assurance stack**: explicit, documented integrity/audit mechanisms beyond a single review pass. For this column, *partial* includes narrower provenance, traceability, automated checking, or human-verifiability mechanisms that do not constitute a full assurance stack. **Cross-platform portability**: skills usable across multiple host environments without re-implementation.

System	Cross-family policy	Adversarial review	Composable skills	E2E Research workflows	Assurance stack	Cross-platform portability
AI Scientist (Lu et al., 2024)	none	partial	×	✓	partial	×
AI Scientist-v2 (Yamada et al., 2025)	none	partial	×	✓	partial	×
Agent Laboratory (Schmidgall et al., 2025)	none	×	×	✓	×	×
data-to-paper (Ifargan et al., 2025)	none	partial	×	✓‡	partial	×
AutoGen (Wu et al., 2023)	none	×	partial	×	×	×
MetaGPT (Hong et al., 2023)	none	partial	partial	×	×	×
OpenHands (Wang et al., 2025; OpenHands, 2026)	none	×	partial	×	×	×
ARIS (ours)	default	✓	✓	✓	✓	✓†

tool—iterative improvement during the writing process—not as a substitute for human peer review.

Harness engineering and agent frameworks. Meta-Harness (Lee et al., 2026) formalizes outer-loop search over harness code; ARIS is a hand-engineered research harness with a prototype outer loop as a step in that direction (§4.5). AutoGen (Wu et al., 2023), CAMEL (Li et al., 2023), OpenHands (Wang et al., 2025), SWE-agent (Yang et al., 2024), MetaGPT (Hong et al., 2023), and ChatDev (Qian et al., 2024) are general-purpose agent or software-engineering frameworks. By contrast, ARIS focuses on research-specific workflows, domain-aware skill definitions, and reviewer-executor separation across model families. Table 4 provides a structured comparison.

8 Conclusion

This report presented ARIS as a research harness built around a conservative assumption: long-horizon research performed by a single agent is unreliable by default, and the relevant failure mode is not visible breakdown but *plausible unsupported success*, where claims outrun evidence and later readers silently inherit the executor's framing. ARIS responds by decomposing the workflow into the three bottlenecks framed in §1—persistent research state, modular execution, and independent assurance—and by adopting a two-role cross-family reviewer-executor pattern as the practical minimum for breaking self-review blind spots. These three bottlenecks map to three layers: an execution layer of reusable Markdown-defined skills and a persistent research wiki, an orchestration layer for configurable workflow control and reviewer routing, and an assurance layer for evidence-to-claim auditing and manuscript checks. A prototype meta-optimization loop provides an initial mechanism for improving skill prompts, defaults, and convergence rules over time.

The main limitations are the absence of controlled evaluation and the reliance on observational deployment evidence. Future work includes compute-matched comparisons to estimate the

contribution of cross-model heterogeneity (Appendix E), local reviewer models for confidential settings, and user studies of researcher productivity.

As a more speculative adjacent direction, the cross-model accountability primitives developed in ARIS—reviewer independence, evidence-to-claim audit, and provenance-aware claim ledgers—are not specific to manuscripts. A natural adaptation is to insert them between any model output and any downstream training-data retention or reward signal, complementing recent self-improvement approaches (Bai et al., 2022; Lee et al., 2023; Yuan et al., 2024; Yu et al., 2025) with an explicit oversight layer. Two known concerns motivate the hypothesis: LLM judges can exhibit systematic biases (Zheng et al., 2023), and recursive training on model-generated data can degrade quality across iterations (Shumailov et al., 2024); cross-family reviewer separation is a candidate mechanism for reducing judge-model coupling, but its downstream effect on long-horizon self-improvement remains an open empirical question. This is a testable future-work hypothesis, not a claim made in this report.

Code and documentation are available at `https://github.com/wanshuiyin/Auto-claude-code-research-in-sleep`.

REFERENCES

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback, 2022. *URL https://arxiv. org/abs/2212.08073*, 2212, 2022.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*, 2024.

Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.

Tal Ifargan, Lukas Hafner, Maor Kern, Ori Alcalay, and Roy Kishony. Autonomous llm-driven research—from data to human-verifiable research papers. *NEJM AI*, 2(1):AIoa2400555, 2025.

Andrej Karpathy. LLM Wiki. GitHub Gist, 2026. URL `https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`. Accessed: 2026-05-03.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, et al. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.

Yu Li, Chenyang Shao, Xinyang Liu, Ruotong Zhao, Peijie Liu, Hongyuan Su, Zhibin Chen, Qinglong Yang, Anjie Xu, Yi Fang, et al. Autosota: An end-to-end automated research system for state-of-the-art ai model discovery. *arXiv preprint arXiv:2604.05550*, 2026.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 17889–17904, 2024a.

Weixin Liang, Yuhui Zhang, Hancheng Cao, Binglu Wang, Daisy Yi Ding, Xinyu Yang, Kailas Vodrahalli, Siyu He, Daniel Scott Smith, Yian Yin, et al. Can large language models provide useful feedback on research papers? a large-scale empirical analysis. *NEJM AI*, 1(8):AIoa2400196, 2024b.

Jiaqi Liu, Peng Xia, Siwei Han, Shi Qiu, Letian Zhang, Guiming Chen, Haoqin Tu, Xinyu Yang, Jiawei Zhou, Hongtu Zhu, Yun Li, Jiaheng Zhang, Yuyin Zhou, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Autoresearchclaw: Fully autonomous research from idea to paper, 2026. URL `https://github.com/aiming-lab/AutoResearchClaw`.

Ryan Liu and Nihar B Shah. Reviewergpt? an exploratory study on using large language models for paper reviewing. *arXiv preprint arXiv:2306.00622*, 2023.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Ziming Luo, Atoosa Kasirzadeh, and Nihar B Shah. The more you automate, the less you see: Hidden pitfalls of ai scientist systems. *arXiv preprint arXiv:2509.08713*, 2025.

Yougang Lyu, Xi Zhang, Xinhao Yi, Yuyue Zhao, Shuyu Guo, Wenxiang Hu, Jan Piotrowski, Jakub Kaliski, Jacopo Urbani, Zaiqiao Meng, Lun Zhou, and Xiaohui Yan. Evoscientist: Towards multi-agent evolving ai scientists for end-to-end scientific discovery. *arXiv preprint arXiv:2603.08127*, 2026.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback, 2023. *URL https://arxiv. org/abs/2303.17651*, 2303, 2023.

OpenHands. OpenHands Skills. Documentation, 2026. URL `https://docs.openhands.dev/overview/skills`. Accessed: 2026-05-03.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 15174–15186, 2024.

Kristin L. Sainani. Writing in the sciences. *Stanford Online Course*, 2019. URL `https://www.coursera.org/learn/sciwrite`. Coursera, Stanford University.

Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 5977–6043, 2025.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. *URL https://arxiv. org/abs/2303.11366*, 8, 2024.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. Ai models collapse when trained on recursively generated data. *Nature*, 631(8022): 755–759, 2024.

Jingqi Tong, Mingzhe Li, Hangcheng Li, Yongzhuo Yang, Yurong Mou, Weijie Ma, Zhiheng Xi, Hongji Chen, Xiaoran Liu, Qinyuan Cheng, et al. Ai can learn scientific taste. *arXiv preprint arXiv:2603.14473*, 2026.

UltraWorkers. Claw Code: Public rust implementation of the claw cli agent harness. GitHub repository, 2026. URL https://github.com/ultraworkers/claw-code. Accessed: 2026-05-03.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=OJd3ayDDoF.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

Tianyu Yu, Haoye Zhang, Qiming Li, Qixin Xu, Yuan Yao, Da Chen, Xiaoman Lu, Ganqu Cui, Yunkai Dang, Taiwen He, et al. Rlaif-v: Open-source ai feedback leads to super gpt-4v trustworthiness. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 19985–19995, 2025.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36: 46595–46623, 2023.

A Workflow Internals

Figures 8–12 show the internal structure of each workflow.

Figure 8: Workflow 1: Idea Discovery. The pipeline surveys literature, brainstorms ideas via cross-model generation, verifies novelty, and refines the top proposal through iterative GPT-5.4 review.

B Skill Inventory

Table 5 lists core framework skills in the current release.

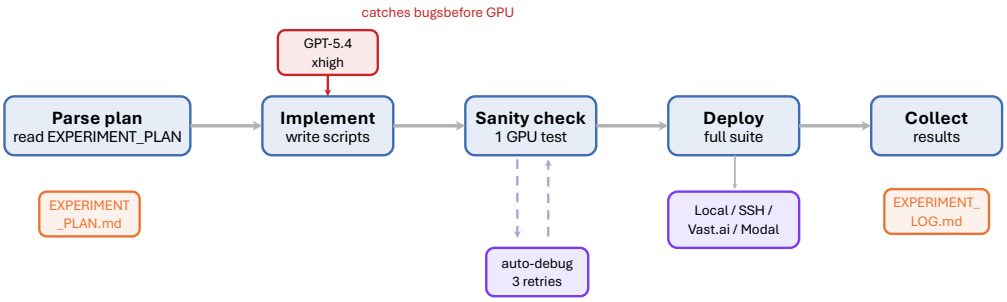


Figure 9: Workflow 1.5: Experiment Bridge. Scripts are implemented, reviewed for code correctness, sanity-checked on one GPU, then deployed to the full backend.

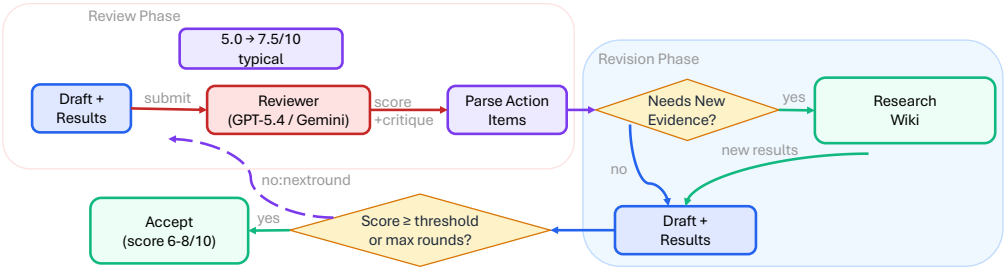


Figure 10: Workflow 2: Auto Review Loop. The reviewer scores the manuscript, the executor implements fixes and runs requested experiments, and the cycle repeats.

C Reviewer Configuration

ARIS configures reviewer behavior along two orthogonal axes (Section 2.2). Table 6 lists the three access-scope settings; Table 7 lists the two context-policy settings.

D ARIS-Code Details

ARIS-Code is a standalone Rust-based CLI built on `claw-code` (UltraWorkers, 2026). Key features: interactive REPL with setup wizard, all skills as slash commands, five LLM providers, native `LlmReview` tool, three-tier skill priority system (user > Claude Code > bundled), and `/cost` for token tracking.

E Controlled Benchmark Protocol (Future Work)

We outline a benchmark protocol for future controlled evaluation: **Task pool**: 12+ paper drafts from publicly available preprints. **Conditions** (compute-matched): (A) single-model self-critique, (B) same-model two-agent, (C) cross-model, (D) cross-model reversed, (E) same-model for the second model. **Metrics**: issue recall, false-positive rate, actionability score, downstream revision quality, cost, latency. **Raters**: three independent, blinded. Inter-rater agreement via Krippendorff's α.

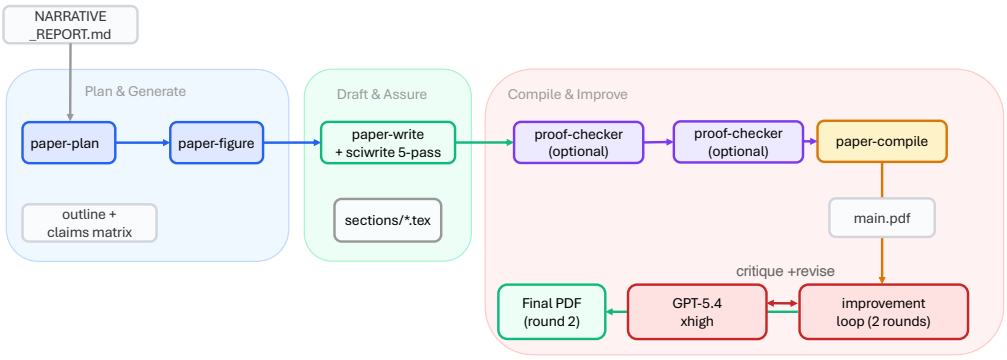


Figure 11: Workflow 3: Paper Writing Pipeline. Seven core sub-skills (plus optional proof checking) chain from outline through figure generation, LaTeX drafting, claim auditing, compilation, and review.

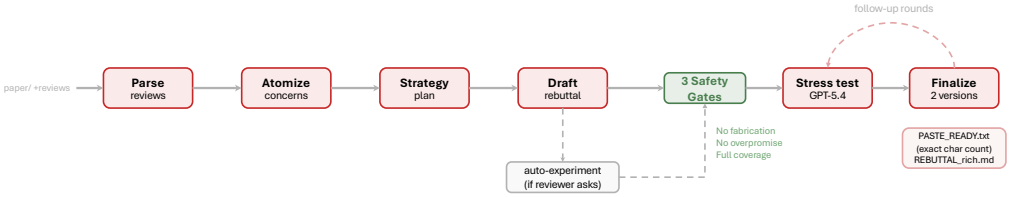


Figure 12: Workflow 4: Rebuttal. Seven phases from parsing reviews through stress-testing, with three safety gates.

Table 5: Core ARIS skill inventory (v0.4, April 2026). Community-contributed skills (30+) are omitted for brevity.

Skill	Category	Reviewer?	Key Function
`research-lit`	Literature	No	Multi-source literature survey
`arxiv` / `alphaxiv`	Literature	No	Paper metadata / LLM-optimized summary
`deepxiv`	Literature	No	Progressive deep literature search
`novelty-check`	Literature	Yes	Novelty verification against existing work
`idea-creator`	Ideation	Yes	Brainstorm and rank research ideas
`idea-discovery`	Ideation	Yes	Full idea discovery pipeline
`experiment-bridge`	Experiment	Yes	Plan to running code
`run-experiment`	Experiment	No	GPU deployment (local/SSH/Vast.ai/Modal)
`monitor-experiment`	Experiment	No	Experiment monitoring and result collection
`check-gpu`	Experiment	No	GPU status and process summary
`analyze-results`	Analysis	No	Statistical analysis and comparison tables
`ablation-planner`	Analysis	No	Reviewer-perspective ablation design
`experiment-audit`	Integrity	Yes	Evaluation code integrity verification
`result-to-claim`	Integrity	No	Result-to-claim verdict conversion
`paper-claim-audit`	Integrity	Yes	Zero-context manuscript number audit
`proof-checker`	Integrity	No	20-category theorem verification
`auto-review-loop`	Review	Yes	Multi-round autonomous review
`research-review`	Review	Yes	GPT-5.4 xhigh deep critique
`paper-plan`	Writing	Yes	Structural outline + claims matrix
`paper-write`	Writing	Yes	Section-by-section LaTeX + sciwrite 5-pass
`paper-figure`	Writing	No	Publication-quality data plots
`paper-compile`	Writing	No	Multi-pass compilation with auto-repair
`paper-writing`	Writing	Yes	Full W3 pipeline orchestrator
`auto-paper-improvement-loop`	Writing	Yes	Review + visual PDF + auto-revision
`rebuttal`	Writing	Yes	7-phase rebuttal with safety gates
`research-wiki`	Memory	No	Persistent knowledge base management
`meta-optimize`	Maintenance	Yes	Outer-loop harness optimization

Table 6: Reviewer access-scope settings (what the reviewer is allowed to read).

Scope	Description
Document-only	Reviewer reads only the manuscript text. Default for paper-writing review.
Artifact-augmented	Reviewer additionally reads result files, claim ledgers, and intermediate artifacts referenced by the manuscript.
Repository-level	Reviewer directly inspects the codebase, evaluation scripts, and generated outputs through host-provided repository access tools (e.g., `claude-code` Bash, `codex exec`). Used by `experiment-audit` and `paper-claim-audit`.

Table 7: Reviewer context-policy settings (whether the reviewer retains state across rounds).

Policy	Description
Fresh	Each review round opens a new reviewer thread with no prior context. Used to prevent confirmation bias from previous rounds (`REVIEWER_BIAS_GUARD = true` default). Required for `paper-claim-audit` and the auto-paper improvement loop.
Cross-round	Reviewer retains memory across rounds, can reference previous critiques and verify whether raised issues have been addressed. Used selectively when convergence verification is more important than independence.